```
# === File: backend/_start_api.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
import os, subprocess, sys
from pathlib import Path
env = os.environ.copy()
p = Path("backend/.env.docker") if Path("backend/.env.docker").exists() else Path(".env.docker")
for line in p.read_text(encoding="utf-8").splitlines():
    line=line.strip()
    if not line or line.startswith("#") or "=" not in line: continue
    k,v=line.split("=",1)
    env[k.strip()]=v.strip()
pu=env.get("POSTGRES_USER","agentflow")
pp=env.get("POSTGRES_PASSWORD","")
pd=env.get("POSTGRES_DB","agentflow_eval")
env["DATABASE_URL"]=f"postgresql+asyncpg://{pu}:{pp}@127.0.0.1:5432/{pd}"
env["REDIS_URL"]="redis://127.0.0.1:6379/0"
env["CELERY_BROKER_URL"]=env["REDIS_URL"]
env["CELERY_RESULT_BACKEND"]=env["REDIS_URL"]
env["CELERY_TASK_ALWAYS_EAGER"]="true"
env["TASK_QUEUE_BACKEND"]="eager"
env["DEPLOY_PROFILE"]="lite"
env["ENV"]="dev"
env["DEBUG"]="true"
env["AUTH_ENABLED"]="false"
env["BILLING_ENABLED"]="false"
env["LOG_DB_SINK"]="true"
env["PYTHONPATH"]=str(Path(".").resolve())
os.chdir(str(Path(".").resolve()))
log=open("uvicorn_docker_db.log","w",encoding="utf-8")
proc=subprocess.Popen([sys.executable,"-m","uvicorn","app.main:app","--host","127.0.0.1","--port","8000"], env=env, stdout=log, stde
print(proc.pid)

# === File: backend/app/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""AgentFlow-Eval 后端应用包。"""

# === File: backend/app/api/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# ? 2026 AgentFlow-Eval

# === File: backend/app/api/v1/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# ? 2026 AgentFlow-Eval

# === File: backend/app/api/v1/endpoints/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# ? 2026 AgentFlow-Eval

# === File: backend/app/api/v1/endpoints/ab.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Online A/B testing REST API."""

from __future__ import annotations

from typing import Any

from fastapi import APIRouter, Depends, Query, Request
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.orm import selectinload

from app.core.ab import service as ab_service
from app.core.ab.service import utcnow
from app.core.audit import write_audit
from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.models.ab_test import ABExperiment, ABStatus, ABVariant
from app.schemas.ab_test import (
    ABAssignRequest,
```

```
    ABAssignResponse,
    ABExperimentCreate,
    ABExperimentListResponse,
    ABExperimentResponse,
    ABSampleSizeRequest,
    ABStatusUpdate,
    ABTrackRequest,
    ABVariantCreate,
    ABVariantResponse,
)
from app.utils.exceptions import BusinessError, NotFoundError

router = APIRouter()

def _actor(request: Request) -> str:
    return getattr(request.state, "actor", None) or "anonymous"

def _to_response(exp: ABExperiment) -> ABExperimentResponse:
    return ABExperimentResponse(
        id=exp.id,
        key=exp.key,
        name=exp.name,
        description=exp.description or "",
        status=exp.status,
        alpha=float(exp.alpha or 0.05),
        min_sample_size=int(exp.min_sample_size or 100),
        primary_metric=exp.primary_metric or "conversion",
        control_variant_key=exp.control_variant_key,
        source_experiment_id=exp.source_experiment_id,
        config=exp.config or {},
        created_by=exp.created_by or "anonymous",
        started_at=exp.started_at,
        ended_at=exp.ended_at,
        winner_variant_key=exp.winner_variant_key,
        created_at=exp.created_at,
        variants=[
            ABVariantResponse(
                id=v.id,
                key=v.key,
                name=v.name or v.key,
                weight=float(v.weight or 1),
                is_control=bool(v.is_control),
                payload=v.payload or {},
                description=v.description or "",
            )
            for v in (exp.variants or [])
        ],
    )

@router.post("", response_model=ABExperimentResponse, status_code=201)
@require_permission(Permission.TASK_CREATE)
async def create_ab_experiment(
    body: ABExperimentCreate,
    request: Request,
```

```
    session: AsyncSession = Depends(get_db),
) -> ABExperimentResponse:
    """Create an online A/B experiment with ≥2 variants."""
    actor = _actor(request)
    existing = (
        await session.execute(select(ABExperiment).where(ABExperiment.key == body.key))
    ).scalar_one_or_none()
    if existing:
        raise BusinessError(f"Experiment key already exists: {body.key}")

    controls = [v for v in body.variants if v.is_control]
    control_key = controls[0].key if controls else body.variants[0].key

    exp = ABExperiment(
        key=body.key,
        name=body.name,
        description=body.description or "",
        status=ABStatus.RUNNING.value
        if body.start_immediately
        else ABStatus.DRAFT.value,
        alpha=body.alpha,
        min_sample_size=body.min_sample_size,
        primary_metric=body.primary_metric,
        control_variant_key=control_key,
        source_experiment_id=body.source_experiment_id,
        config=body.config or {},
        created_by=actor,
        started_at=utcnow() if body.start_immediately else None,
    )
    session.add(exp)
    await session.flush()

    for i, v in enumerate(body.variants):
        is_ctrl = v.is_control or (not controls and i == 0)
        session.add(
            ABVariant(
                experiment_id=exp.id,
                key=v.key,
                name=v.name or v.key,
                weight=v.weight,
                is_control=is_ctrl,
                payload=v.payload or {},
                description=v.description or "",
            )
        )

    await write_audit(
        session,
        action="ab.create",
        resource_type="ab_experiment",
        resource_id=exp.id,
        actor=actor,
        detail={"key": body.key, "variants": [v.key for v in body.variants]},
    )
```

```
    await session.commit()
    exp = await ab_service.get_experiment_by_id(session, exp.id)
    return _to_response(exp)

@router.get("", response_model=ABExperimentListResponse)
@require_permission(Permission.TASK_READ)
async def list_ab_experiments(
    request: Request,
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    status: str | None = Query(None),
    session: AsyncSession = Depends(get_db),
) -> ABExperimentListResponse:
    actor = _actor(request)
    q = select(ABExperiment).options(selectinload(ABExperiment.variants))
    cq = select(func.count(ABExperiment.id))
    from app.core.tenancy import is_admin, tenancy_enforced

    if tenancy_enforced() and not is_admin(actor):
        q = q.where(ABExperiment.created_by == actor)
        cq = cq.where(ABExperiment.created_by == actor)
    if status:
        q = q.where(ABExperiment.status == status)
        cq = cq.where(ABExperiment.status == status)
    total = int((await session.execute(cq)).scalar() or 0)
    rows = (
        (
            await session.execute(
                q.order_by(ABExperiment.created_at.desc())
                .offset((page - 1) * page_size)
                .limit(page_size)
            )
        )
        .scalars()
        .all()
    )
    return ABExperimentListResponse(
        items=[_to_response(r) for r in rows],
        total=total,
        page=page,
        page_size=page_size,
    )

@router.post("/sample-size")
@require_permission(Permission.TASK_READ)
async def sample_size(
    body: ABSampleSizeRequest,
    request: Request,
) -> dict[str, Any]:
    """Recommend per-variant sample size for conversion-rate A/B tests."""
    return await ab_service.recommend_sample_size(
        baseline_rate=body.baseline_rate,
        mde=body.mde,
```

```
        alpha=body.alpha,
        power=body.power,
    )

@router.post(
    "/from-offline/{experiment_id}",
    response_model=ABExperimentResponse,
    status_code=201,
)
@require_permission(Permission.TASK_CREATE)
async def create_ab_from_offline_experiment(
    experiment_id: str,
    request: Request,
    key: str = Query(..., min_length=1, max_length=100),
    session: AsyncSession = Depends(get_db),
) -> ABExperimentResponse:
    """Promote an offline Experiment's runs into an online A/B draft."""
    from app.models.experiment import Experiment

    actor = _actor(request)
    offline = (
        await session.execute(
            select(Experiment)
            .options(selectinload(Experiment.runs))
            .where(Experiment.id == experiment_id)
        )
    ).scalar_one_or_none()
    if offline is None:
        raise NotFoundError("Experiment", experiment_id)
    if not offline.runs or len(offline.runs) < 2:
        raise BusinessError("Offline experiment needs at least 2 runs to create A/B")

    existing = (
        await session.execute(select(ABExperiment).where(ABExperiment.key == key))
    ).scalar_one_or_none()
    if existing:
        raise BusinessError(f"Experiment key already exists: {key}")

    variants = [
        ABVariantCreate(
            key=r.label,
            name=r.label,
            weight=1.0,
            is_control=(i == 0),
            payload={"agent_config": r.agent_config or {}},
        )
        for i, r in enumerate(offline.runs)
    ]
    exp = ABExperiment(
        key=key,
        name=f"AB from {offline.name}",
        description=f"Promoted from offline experiment {offline.id}",
        status=ABStatus.DRAFT.value,
        control_variant_key=variants[0].key,
```

```
        source_experiment_id=offline.id,
        created_by=actor,
        config={},
    )
    session.add(exp)
    await session.flush()
    for i, v in enumerate(variants):
        session.add(
            ABVariant(
                experiment_id=exp.id,
                key=v.key,
                name=v.name,
                weight=v.weight,
                is_control=v.is_control or i == 0,
                payload=v.payload,
            )
        )
    await session.commit()
    exp = await ab_service.get_experiment_by_id(session, exp.id)
    return _to_response(exp)

@router.get("/{experiment_id}", response_model=ABExperimentResponse)
@require_permission(Permission.TASK_READ)
async def get_ab_experiment(
    experiment_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> ABExperimentResponse:
    exp = await ab_service.get_experiment_by_id(session, experiment_id)
    return _to_response(exp)

@router.patch("/{experiment_id}/status", response_model=ABExperimentResponse)
@require_permission(Permission.TASK_UPDATE)
async def update_ab_status(
    experiment_id: str,
    body: ABStatusUpdate,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> ABExperimentResponse:
    exp = await ab_service.get_experiment_by_id(session, experiment_id)
    new_status = body.status
    if new_status == ABStatus.RUNNING.value and exp.status != ABStatus.RUNNING.value:
        exp.started_at = exp.started_at or utcnow()
        exp.ended_at = None
    if new_status in {ABStatus.COMPLETED.value, ABStatus.ARCHIVED.value}:
        exp.ended_at = utcnow()
    exp.status = new_status
    await session.commit()
    exp = await ab_service.get_experiment_by_id(session, experiment_id)
    return _to_response(exp)

@router.post("/{experiment_key}/assign", response_model=ABAssignResponse)
@require_permission(Permission.TASK_READ)
```

```
async def assign_unit(
    experiment_key: str,
    body: ABAssignRequest,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> ABAssignResponse:
    """Sticky assign a unit to a variant (and optionally log exposure)."""
    payload = await ab_service.assign(
        session,
        experiment_key=experiment_key,
        unit_id=body.unit_id,
        context=body.context,
        record_exposure=body.record_exposure,
    )
    return ABAssignResponse(**payload)

@router.post("/{experiment_key}/track")
@require_permission(Permission.TASK_UPDATE)
async def track_event(
    experiment_key: str,
    body: ABTrackRequest,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Track exposure / conversion / metric for a unit."""
    return await ab_service.track_event(
        session,
        experiment_key=experiment_key,
        unit_id=body.unit_id,
        event_type=body.event_type,
        metric_name=body.metric_name,
        metric_value=body.metric_value,
        properties=body.properties,
        auto_assign=body.auto_assign,
    )

@router.get("/{experiment_id}/results")
@require_permission(Permission.EVALUATION_READ)
async def ab_results(
    experiment_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Statistical analysis: conversion rates, lift, p-values, optional winner."""
    return await ab_service.analyze(session, experiment_id)

# === File: backend/app/api/v1/endpoints/agents_http.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""HTTP Agent product APIs — contract docs + probe (Phase 1)."""

from __future__ import annotations

import time
from typing import Any

import httpx
from fastapi import APIRouter, Request
```

```
from pydantic import BaseModel, Field

from app.config import settings
from app.core.agent_runner.protocol import (
    PROTOCOL_VERSION,
    build_http_request_payload,
    coerce_http_response,
    failed_http_result,
)
from app.core.agent_runner.ssrf import SsrfBlockedError, validate_http_agent_url
from app.core.rbac import Permission, require_permission

router = APIRouter()

class HttpAgentProbeRequest(BaseModel):
    """Probe an external HTTP agent endpoint."""

    endpoint_url: str = Field(..., min_length=1, description="Absolute http(s) URL")
    timeout_sec: float = Field(default=10.0, ge=1.0, le=120.0)
    headers: dict[str, str] = Field(default_factory=dict)
    method: str = Field(default="POST")
    query: str = Field(default="ping", description="Probe query string")
    context: dict[str, Any] = Field(default_factory=dict)
    verify_ssl: bool = True

class HttpAgentProbeResponse(BaseModel):
    """Probe result — always JSON (use ok/reachable for UX, not only HTTP status)."""

    ok: bool = False
    reachable: bool = False
    protocol_compatible: bool = False
    ssrf_blocked: bool = False
    latency_ms: int | None = None
    http_status: int | None = None
    endpoint: str = ""
    protocol_version: str = PROTOCOL_VERSION
    final_answer_preview: str = ""
    steps_count: int = 0
    normalized_status: str = ""
    error: str | None = None
    detail: dict[str, Any] = Field(default_factory=dict)

@router.get("/contract")
@require_permission(Permission.TASK_READ)
async def http_agent_contract(request: Request) -> dict[str, Any]:
    """Return agentflow.http.v1 contract summary for UI / integrators."""
    return {
        "protocol_version": PROTOCOL_VERSION,
        "request": {
            "method": "POST",
            "content_type": "application/json",
            "body": {
                "protocol_version": PROTOCOL_VERSION,
                "query": "string",
                "tools": ["string"],
                "context": {},
                "meta": {},
```

```
            },
        },
        "response_accepted": [
            {
                "name": "full",
                "example": {
                    "status": "success",
                    "final_answer": "...",
                    "steps": [],
                    "total_tokens": 0,
                    "response_time_ms": 0,
                },
            },
            {"name": "short", "example": {"answer": "..."}},
            {"name": "plain_text", "example": "raw answer body"},
        ],
        "agent_config_fields": {
            "runner": "http | http_agent | remote | webhook",
            "endpoint_url": "https://agent.example.com/v1/invoke",
            "timeout_sec": 60,
            "headers": {"Authorization": "Bearer ..."},
            "method": "POST",
            "context": {},
            "verify_ssl": True,
        },
        "docs": "docs/http-agent-protocol.md",
    }

@router.post("/probe", response_model=HttpAgentProbeResponse)
@require_permission(Permission.TASK_CREATE, Permission.TASK_EXECUTE)
async def probe_http_agent(
    body: HttpAgentProbeRequest,
    request: Request,
) -> HttpAgentProbeResponse:
    """Probe an external HTTP agent: SSRF check, reachability, protocol normalize."""
    allow_private = bool(getattr(settings, "HTTP_AGENT_ALLOW_PRIVATE_IP", False))
    endpoint = (body.endpoint_url or "").strip()

    try:
        endpoint = validate_http_agent_url(endpoint, allow_private=allow_private)
    except SsrfBlockedError as exc:
        return HttpAgentProbeResponse(
            ok=False,
            reachable=False,
            protocol_compatible=False,
            ssrf_blocked=True,
            endpoint=endpoint or body.endpoint_url,
            error=str(exc),
            detail={"code": "ssrf_blocked"},
        )

    method = (body.method or "POST").upper()
    payload = build_http_request_payload(
```

```
        body.query or "ping",
        tools=[],
        context=body.context or {},
        meta={"probe": True},
    )
    headers = {"Content-Type": "application/json", **(body.headers or {})}

    t0 = time.monotonic()
    try:
        async with httpx.AsyncClient(
            timeout=float(body.timeout_sec),
            verify=bool(body.verify_ssl),
        ) as client:
            resp = await client.request(
                method,
                endpoint,
                json=payload if method in {"POST", "PUT", "PATCH"} else None,
                params=payload if method == "GET" else None,
                headers=headers,
            )
        latency = int((time.monotonic() - t0) * 1000)
    except httpx.TimeoutException as exc:
        return HttpAgentProbeResponse(
            ok=False,
            reachable=False,
            protocol_compatible=False,
            latency_ms=int((time.monotonic() - t0) * 1000),
            endpoint=endpoint,
            error=f"Timeout after {body.timeout_sec}s: {exc}",
            detail={"code": "timeout"},
        )
    except httpx.HTTPError as exc:
        return HttpAgentProbeResponse(
            ok=False,
            reachable=False,
            protocol_compatible=False,
            latency_ms=int((time.monotonic() - t0) * 1000),
            endpoint=endpoint,
            error=f"HTTP request failed: {exc}",
            detail={"code": "http_error"},
        )

    if resp.status_code >= 400:
        normalized = failed_http_result(
            query=body.query or "ping",
            error=f"HTTP {resp.status_code}: {(resp.text or '')[:300]}",
            elapsed_ms=latency,
            endpoint=endpoint,
        )
        return HttpAgentProbeResponse(
            ok=False,
            reachable=True,
```

```
            protocol_compatible=False,
            latency_ms=latency,
            http_status=resp.status_code,
            endpoint=endpoint,
            error=normalized.get("error_message") or f"HTTP {resp.status_code}",
            normalized_status="failed",
            detail={"code": "http_status", "body_preview": (resp.text or "")[:200]},
        )

    ct = (resp.headers.get("content-type") or "").lower()
    data: Any
    if "application/json" in ct or (resp.text or "").lstrip().startswith(("{", "["):
        try:
            data = resp.json()
        except ValueError:
            data = {"answer": resp.text}
    else:
        data = {"answer": resp.text or ""}

    normalized = coerce_http_response(
        data, query=body.query or "ping", elapsed_ms=latency, endpoint=endpoint
    )
    status = str(normalized.get("status") or "")
    final = str(normalized.get("final_answer") or "")
    steps = normalized.get("steps") or []
    compatible = status in {"success", "max_iterations_reached"} and (
        bool(final) or bool(steps)
    )
    parseable = bool(normalized.get("runner") == "http")

    return HttpAgentProbeResponse(
        ok=compatible,
        reachable=True,
        protocol_compatible=compatible or parseable,
        latency_ms=latency,
        http_status=resp.status_code,
        endpoint=endpoint,
        final_answer_preview=final[:200],
        steps_count=len(steps) if isinstance(steps, list) else 0,
        normalized_status=status,
        error=None if compatible else (normalized.get("error_message") or "No usable answer"),
        detail={
            "code": "ok" if compatible else "protocol_partial",
            "total_tokens": normalized.get("total_tokens"),
        },
    )

# === File: backend/app/api/v1/endpoints/audit.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Audit log query API."""

from __future__ import annotations

from typing import Any

from fastapi import APIRouter, Depends, Query, Request
from sqlalchemy import func, select
from sqlalchemy.ext.asyncio import AsyncSession
```

```
from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.models.audit_log import AuditLog

router = APIRouter()

@router.get("")
@require_permission(Permission.AUDIT_READ)
async def list_audit_logs(
    request: Request,
    page: int = Query(1, ge=1),
    page_size: int = Query(20, ge=1, le=100),
    action: str | None = Query(None, description="Filter by action"),
    resource_type: str | None = Query(None),
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """List recent audit events (newest first). Requires audit:read."""
    query = select(AuditLog)
    count_query = select(func.count(AuditLog.id))

    if action:
        query = query.where(AuditLog.action == action)
        count_query = count_query.where(AuditLog.action == action)
    if resource_type:
        query = query.where(AuditLog.resource_type == resource_type)
        count_query = count_query.where(AuditLog.resource_type == resource_type)

    total = (await session.execute(count_query)).scalar() or 0
    result = await session.execute(
        query.order_by(AuditLog.created_at.desc())
        .offset((page - 1) * page_size)
        .limit(page_size)
    )
    rows = result.scalars().all()
    items = [
        {
            "id": r.id,
            "actor": r.actor,
            "action": r.action,
            "resource_type": r.resource_type,
            "resource_id": r.resource_id,
            "detail": r.detail,
            "request_id": r.request_id,
            "ip_address": r.ip_address,
            "created_at": r.created_at.isoformat() if r.created_at else None,
        }
        for r in rows
    ]
    return {"items": items, "total": total, "page": page, "page_size": page_size}

# === File: backend/app/api/v1/endpoints/benchmarks.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Benchmark platform API."""

from __future__ import annotations

from typing import Any

from fastapi import APIRouter, Depends, File, Query, Request, UploadFile
```

```
from pydantic import BaseModel, Field
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.benchmark.service import get_benchmark_service
from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.core.tenant_context import (
    current_tenant_id,
    extract_tenant_header,
    resolve_tenant_context,
)
from app.utils.exceptions import AgentFlowError

router = APIRouter()

def _actor(request: Request) -> str:
    return getattr(request.state, "actor", None) or "anonymous"

async def _bind_tenant(request: Request, session: AsyncSession) -> str | None:
    role = getattr(request.state, "role", None)
    role_s = role.value if hasattr(role, "value") else str(role or "")
    await resolve_tenant_context(
        session,
        actor=_actor(request),
        header_value=extract_tenant_header(request),
        system_role=role_s,
    )
    return current_tenant_id()

class BenchmarkCreate(BaseModel):
    name: str = Field(..., min_length=1, max_length=255)
    description: str = ""
    tags: list[str] = Field(default_factory=list)
    cases: list[dict[str, Any]] = Field(default_factory=list)
    version: str = Field(default="1.0", max_length=64)
    scorecard: dict[str, Any] | None = Field(
        default=None, description="Optional Scorecard JSON (Phase 3)"
    )
    source_task_id: str | None = Field(
        default=None, description="If set, clone suites from this Task as cases"
    )

class BenchmarkRunBody(BaseModel):
    label: str = "default"
    agent_config: dict[str, Any] = Field(
        default_factory=lambda: {
            "runner": "openai",
            "model": "gpt-4o-mini",
            "temperature": 0,
        }
    )
    enqueue: bool = True

class ImportBody(BaseModel):
    format: str = Field("json", description="json | csv")
    content: str = Field(..., min_length=1)

class CompareRunsBody(BaseModel):
```

```
    """Compare current run against a baseline (defaults to previous completed run)."""

    current_run_id: str
    baseline_run_id: str | None = Field(
        default=None,
        description="If omitted, use the previous completed run before current",
    )
    score_stable_eps: float = Field(
        default=1.0, ge=0, description="|Δscore| below this → stable"
    )

def _bm_dict(bm, *, case_count: int | None = None) -> dict[str, Any]:
    meta = dict(bm.meta or {})
    return {
        "id": bm.id,
        "name": bm.name,
        "description": bm.description,
        "status": bm.status,
        "created_by": bm.created_by,
        "tags": bm.tags or [],
        "tenant_id": bm.tenant_id,
        "version": meta.get("version") or "1.0",
        "scorecard": meta.get("scorecard"),
        "source_task_id": meta.get("source_task_id"),
        "case_count": case_count
        if case_count is not None
        else (len(bm.cases) if getattr(bm, "cases", None) is not None else None),
        "created_at": bm.created_at.isoformat() if bm.created_at else None,
    }

def _run_dict(run) -> dict[str, Any]:
    return {
        "id": run.id,
        "benchmark_id": run.benchmark_id,
        "task_id": run.task_id,
        "label": run.label,
        "status": run.status,
        "agent_config": run.agent_config or {},
        "summary": run.summary or {},
        "created_by": run.created_by,
        "created_at": run.created_at.isoformat() if run.created_at else None,
        "started_at": run.started_at.isoformat() if run.started_at else None,
        "finished_at": run.finished_at.isoformat() if run.finished_at else None,
    }

@router.get("")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def list_benchmarks(
    request: Request,
    limit: int = Query(50, ge=1, le=200),
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    tenant_id = await _bind_tenant(request, session)
    svc = get_benchmark_service()
```

```
    items = await svc.list_benchmarks(
        session, actor=_actor(request), tenant_id=tenant_id, limit=limit
    )
    out = []
    for bm in items:
        full = await svc.get_benchmark(session, bm.id, with_cases=True)
        out.append(_bm_dict(full, case_count=len(full.cases)))
    return {"items": out, "total": len(out)}

@router.post("", status_code=201)
@require_permission(
    Permission.BENCHMARK_CREATE, Permission.TASK_CREATE, require_all=False
)
async def create_benchmark(
    body: BenchmarkCreate,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    tenant_id = await _bind_tenant(request, session)
    svc = get_benchmark_service()
    if body.source_task_id:
        bm = await svc.create_from_task(
            session,
            task_id=body.source_task_id,
            name=body.name,
            description=body.description,
            version=body.version,
            created_by=_actor(request),
            tenant_id=tenant_id,
            scorecard=body.scorecard,
        )
        if body.cases:
            await svc.add_cases(
                session, bm, body.cases, tenant_id=tenant_id
            )
    else:
        bm = await svc.create_benchmark(
            session,
            name=body.name,
            description=body.description,
            created_by=_actor(request),
            tenant_id=tenant_id,
            tags=body.tags,
            cases=body.cases,
            version=body.version,
            scorecard=body.scorecard,
        )
    await session.commit()
    full = await svc.get_benchmark(session, bm.id, with_cases=True)
    return {
        **_bm_dict(full, case_count=len(full.cases)),
```

```
        "cases": [
            {
                "id": c.id,
                "name": c.name,
                "user_query": c.user_query,
                "expected_output": c.expected_output,
                "expected_tools": c.expected_tools,
                "weight": c.weight,
            }
            for c in full.cases
        ],
    }

@router.get("/{benchmark_id}")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def get_benchmark(
    benchmark_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    await _bind_tenant(request, session)
    svc = get_benchmark_service()
    bm = await svc.get_benchmark(session, benchmark_id, with_cases=True)
    return {
        **_bm_dict(bm, case_count=len(bm.cases)),
        "cases": [
            {
                "id": c.id,
                "name": c.name,
                "user_query": c.user_query,
                "expected_output": c.expected_output,
                "expected_tools": c.expected_tools,
                "weight": c.weight,
            }
            for c in bm.cases
        ],
    }

@router.post("/{benchmark_id}/import")
@require_permission(
    Permission.BENCHMARK_CREATE, Permission.TASK_CREATE, require_all=False
)
async def import_cases(
    benchmark_id: str,
    body: ImportBody,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    tenant_id = await _bind_tenant(request, session)
    svc = get_benchmark_service()
    bm = await svc.get_benchmark(session, benchmark_id, with_cases=False)
    try:
```

```
        cases = svc.parse_import_payload(content=body.content, fmt=body.format)
    except AgentFlowError:
        raise
    except Exception as exc:
        raise AgentFlowError(f"import parse failed: {exc}", status_code=422) from exc
    created = await svc.add_cases(session, bm, cases, tenant_id=tenant_id)
    await session.commit()
    return {"imported": len(created), "benchmark_id": bm.id}

@router.post("/{benchmark_id}/import/file")
@require_permission(
    Permission.BENCHMARK_CREATE, Permission.TASK_CREATE, require_all=False
)
async def import_cases_file(
    benchmark_id: str,
    request: Request,
    file: UploadFile = File(...),
    format: str = Query("json", description="json | csv"),
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    tenant_id = await _bind_tenant(request, session)
    raw = await file.read()
    content = raw.decode("utf-8", errors="replace")
    name = (file.filename or "").lower()
    fmt = format
    if name.endswith(".csv"):
        fmt = "csv"
    elif name.endswith(".json"):
        fmt = "json"
    svc = get_benchmark_service()
    bm = await svc.get_benchmark(session, benchmark_id, with_cases=False)
    cases = svc.parse_import_payload(content=content, fmt=fmt)
    created = await svc.add_cases(session, bm, cases, tenant_id=tenant_id)
    await session.commit()
    return {"imported": len(created), "benchmark_id": bm.id, "format": fmt}

@router.post("/{benchmark_id}/run", status_code=201)
@require_permission(
    Permission.BENCHMARK_CREATE, Permission.TASK_EXECUTE, require_all=False
)
async def run_benchmark(
    benchmark_id: str,
    body: BenchmarkRunBody,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Run benchmark through Evaluation Engine (creates Task + enqueues)."""
    tenant_id = await _bind_tenant(request, session)
    svc = get_benchmark_service()
    try:
        run = await svc.run_benchmark(
            session,
```

```
            benchmark_id=benchmark_id,
            actor=_actor(request),
            agent_config=body.agent_config,
            label=body.label,
            tenant_id=tenant_id,
            enqueue=body.enqueue,
        )
    except AgentFlowError:
        raise
    await session.commit()
    return {"run": _run_dict(run)}

@router.get("/{benchmark_id}/runs")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def list_runs(
    benchmark_id: str,
    request: Request,
    limit: int = Query(50, ge=1, le=200),
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """History of regression evaluation runs for continuous evaluation."""
    await _bind_tenant(request, session)
    svc = get_benchmark_service()
    runs = await svc.list_runs(session, benchmark_id, limit=limit)
    await session.commit()
    return {
        "benchmark_id": benchmark_id,
        "items": [_run_dict(r) for r in runs],
        "total": len(runs),
    }

@router.post("/{benchmark_id}/runs/{run_id}/finalize")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def finalize_run(
    benchmark_id: str,
    run_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    await _bind_tenant(request, session)
    svc = get_benchmark_service()
    run = await svc.finalize_run(session, run_id)
    if run.benchmark_id != benchmark_id:
        raise AgentFlowError("Run does not belong to benchmark", status_code=400)
    await session.commit()
    return {"run": _run_dict(run)}

@router.post("/{benchmark_id}/compare")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def compare_runs(
    benchmark_id: str,
    body: CompareRunsBody,
    request: Request,
```

```
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Compare two runs (or current vs previous) for regression detection."""
    await _bind_tenant(request, session)
    svc = get_benchmark_service()
    await svc.get_benchmark(session, benchmark_id, with_cases=False)

    current = await svc.get_run(session, body.current_run_id)
    if current.benchmark_id != benchmark_id:
        raise AgentFlowError("current_run_id not in this benchmark", status_code=400)

    if body.baseline_run_id:
        baseline = await svc.get_run(session, body.baseline_run_id)
        if baseline.benchmark_id != benchmark_id:
            raise AgentFlowError(
                "baseline_run_id not in this benchmark", status_code=400
            )
    else:
        history = await svc.list_runs(session, benchmark_id, limit=100)
        baseline = None
        for r in history:
            if r.id == current.id:
                continue
            if r.created_at and current.created_at and r.created_at >= current.created_at:
                continue
            if r.status == "completed" or (r.summary or {}).get("score") is not None:
                baseline = r
                break
        if baseline is None:
            raise AgentFlowError(
                "No baseline run found — provide baseline_run_id or complete a prior run",
                status_code=400,
            )

    result = svc.compare_runs(
        current, baseline, score_stable_eps=body.score_stable_eps
    )
    await session.commit()
    return {"benchmark_id": benchmark_id, **result}

@router.get("/{benchmark_id}/leaderboard")
@require_permission(Permission.BENCHMARK_READ, Permission.TASK_READ, require_all=False)
async def leaderboard(
    benchmark_id: str,
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    await _bind_tenant(request, session)
    svc = get_benchmark_service()
    await svc.get_benchmark(session, benchmark_id, with_cases=False)
    board = await svc.leaderboard(session, benchmark_id)
    await session.commit()
    return {
        "benchmark_id": benchmark_id,
```

```
        "items": board,
        "total": len(board),
        "metrics": [
            "score",
            "success_rate",
            "score_coverage",
            "accuracy",
            "quality",
            "latency_ms",
            "cost",
            "tokens",
        ],
    }

# === File: backend/app/api/v1/endpoints/billing.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Billing / subscription / usage APIs (BILLING_ENABLED optional)."""

from __future__ import annotations

import json
from typing import Any

from fastapi import APIRouter, Depends, Query, Request  # Query used by rollover
from pydantic import BaseModel, Field
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.billing.service import billing_enabled, get_billing_service
from app.core.dependencies import get_db
from app.core.rbac import Permission, require_permission
from app.utils.exceptions import AgentFlowError

router = APIRouter()

def _actor(request: Request) -> str:
    return getattr(request.state, "actor", None) or "anonymous"

class SubscribeBody(BaseModel):
    plan_code: str = Field(..., description="free | pro | enterprise")

def _plan_dict(p) -> dict[str, Any]:
    return {
        "id": p.id,
        "code": p.code,
        "name": p.name,
        "description": p.description,
        "price_month_cents": p.price_month_cents,
        "billing_cycle": getattr(p, "billing_cycle", None) or "monthly",
        "token_quota": p.token_quota,
        "task_quota": p.task_quota,
        "storage_quota_mb": getattr(p, "storage_quota_mb", None),
        "plugin_quota": getattr(p, "plugin_quota", None),
        "features": p.features or {},
        "limits": (p.features or {}).get("limits")
        or {
            "tasks": p.task_quota,
            "tokens": p.token_quota,
            "storage_mb": getattr(p, "storage_quota_mb", None),
            "plugins": getattr(p, "plugin_quota", None),
        },
```

```
        "is_public": p.is_public,
    }

@router.get("/plans")
@require_permission(Permission.BILLING_READ, Permission.TASK_READ, require_all=False)
async def list_plans(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    svc = get_billing_service()
    await svc.ensure_default_plans(session)
    plans = await svc.list_plans(session, public_only=True)
    return {
        "items": [_plan_dict(p) for p in plans],
        "total": len(plans),
        "billing_enabled": billing_enabled(),
    }

@router.get("/plan")
@require_permission(Permission.BILLING_READ, Permission.TASK_READ, require_all=False)
async def get_current_plan(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Current actor plan + quota (enterprise contract alias)."""
    actor = _actor(request)
    svc = get_billing_service()
    return await svc.get_current_plan(session, actor)

@router.get("/quota")
@require_permission(Permission.BILLING_READ, Permission.TASK_READ, require_all=False)
async def get_quota(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    actor = _actor(request)
    svc = get_billing_service()
    return await svc.get_quota(session, actor)

@router.get("/usage")
@require_permission(Permission.BILLING_READ, Permission.TASK_READ, require_all=False)
async def list_usage(
    request: Request,
    limit: int = Query(50, ge=1, le=500),
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    actor = _actor(request)
    svc = get_billing_service()
    rows = await svc.list_usage(session, actor, limit=limit)
    return {
        "items": [
            {
                "id": r.id,
                "metric": r.metric,
```

```
                "quantity": r.quantity,
                "ref_type": r.ref_type,
                "ref_id": r.ref_id,
                "trace_id": r.trace_id,
                "created_at": r.created_at.isoformat() if r.created_at else None,
            }
            for r in rows
        ],
        "total": len(rows),
        "billing_enabled": billing_enabled(),
    }

class CheckoutBody(BaseModel):
    plan_code: str = Field(..., description="pro | enterprise (not free)")
    success_url: str | None = None
    cancel_url: str | None = None

class MockConfirmBody(BaseModel):
    session_id: str
    plan_code: str
    actor: str | None = None

@router.post("/subscribe")
@require_permission(
    Permission.BILLING_MANAGE, Permission.SYSTEM_CONFIG, require_all=False
)
async def subscribe(
    request: Request,
    body: SubscribeBody,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Direct subscribe (ops / free tier / mock without payment).

    Paid plans in production should use ``POST /billing/checkout``.
    """
    actor = _actor(request)
    svc = get_billing_service()
    await svc.ensure_default_plans(session)
    try:
        sub = await svc.subscribe(session, actor=actor, plan_code=body.plan_code)
    except AgentFlowError:
        raise
    except Exception as exc:
        raise AgentFlowError(f"subscribe failed: {exc}", status_code=400) from exc
    await session.commit()
    return {
        "subscription": {
            "id": sub.id,
            "actor": sub.actor,
            "plan_id": sub.plan_id,
            "status": sub.status,
        }
    }

@router.post("/checkout")
```

```
@require_permission(
    Permission.BILLING_MANAGE, Permission.SYSTEM_CONFIG, require_all=False
)
async def create_checkout(
    request: Request,
    body: CheckoutBody,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Create Stripe Checkout session (mock by default).

    Returns ``url`` to redirect the browser. In mock mode, complete with
    ``POST /billing/checkout/mock-confirm``.
    """
    from app.core.billing.stripe_checkout import create_checkout_session, stripe_mode

    actor = _actor(request)
    svc = get_billing_service()
    await svc.ensure_default_plans(session)
    plan = await svc.get_plan_by_code(session, body.plan_code)
    if plan is None:
        raise AgentFlowError(f"Unknown plan: {body.plan_code}", status_code=404)
    if plan.code == "free" or plan.price_month_cents <= 0:
        sub = await svc.subscribe(session, actor=actor, plan_code="free")
        await session.commit()
        return {
            "mode": "direct",
            "url": None,
            "subscription": {
                "id": sub.id,
                "actor": sub.actor,
                "plan_id": sub.plan_id,
                "status": sub.status,
            },
        }
    try:
        session_out = create_checkout_session(
            actor=actor,
            plan_code=plan.code,
            plan_name=plan.name,
            amount_cents=plan.price_month_cents,
            success_url=body.success_url,
            cancel_url=body.cancel_url,
        )
    except Exception as exc:
        raise AgentFlowError(f"checkout failed: {exc}", status_code=400) from exc
    return {
        "checkout": session_out,
        "stripe_mode": stripe_mode(),
        "billing_enabled": billing_enabled(),
    }

@router.post("/checkout/mock-confirm")
@require_permission(
```

```
    Permission.BILLING_MANAGE, Permission.SYSTEM_CONFIG, require_all=False
)
async def mock_confirm_checkout(
    request: Request,
    body: MockConfirmBody,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Complete a mock Checkout session and activate the subscription."""
    from app.core.billing.stripe_checkout import (
        build_mock_completed_event,
        parse_checkout_completed_event,
        stripe_mode,
    )

    if stripe_mode() != "mock":
        raise AgentFlowError(
            "mock-confirm only available when STRIPE_MODE=mock",
            status_code=400,
        )
    actor = body.actor or _actor(request)
    event = build_mock_completed_event(
        actor=actor,
        plan_code=body.plan_code,
        session_id=body.session_id,
    )
    parsed = parse_checkout_completed_event(event)
    if not parsed:
        raise AgentFlowError("invalid mock session", status_code=400)
    svc = get_billing_service()
    await svc.ensure_default_plans(session)
    try:
        sub = await svc.subscribe(
            session, actor=parsed["actor"], plan_code=parsed["plan_code"]
        )
        sub.external_ref = parsed.get("external_ref") or body.session_id
    except Exception as exc:
        raise AgentFlowError(f"activate failed: {exc}", status_code=400) from exc
    await session.commit()
    return {
        "ok": True,
        "mode": "mock",
        "subscription": {
            "id": sub.id,
            "actor": sub.actor,
            "plan_id": sub.plan_id,
            "status": sub.status,
            "external_ref": sub.external_ref,
        },
    }

async def _handle_stripe_webhook(
    request: Request,
```

```
    session: AsyncSession,
) -> dict[str, Any]:
    """Shared Stripe/mock webhook body (public — signature verified)."""
    from app.core.billing.stripe_checkout import (
        parse_checkout_completed_event,
        verify_webhook_signature,
    )

    payload = await request.body()
    sig = request.headers.get("stripe-signature") or request.headers.get(
        "Stripe-Signature"
    )
    if not verify_webhook_signature(payload, sig):
        raise AgentFlowError("invalid webhook signature", status_code=400)

    try:
        event = json.loads(payload.decode("utf-8") or "{}")
    except Exception as exc:
        raise AgentFlowError(f"invalid JSON: {exc}", status_code=400) from exc

    parsed = parse_checkout_completed_event(event)
    if not parsed:
        return {"received": True, "handled": False, "reason": "ignored_event"}

    svc = get_billing_service()
    await svc.ensure_default_plans(session)
    try:
        sub = await svc.subscribe(
            session,
            actor=parsed["actor"],
            plan_code=parsed["plan_code"],
        )
        if parsed.get("external_ref"):
            sub.external_ref = parsed["external_ref"]
        await session.commit()
    except Exception as exc:
        await session.rollback()
        raise AgentFlowError(
            f"webhook activate failed: {exc}", status_code=400
        ) from exc

    try:
        from app.core.audit import write_audit

        await write_audit(
            session,
            action="billing.checkout_completed",
            resource_type="subscription",
            resource_id=sub.id,
            actor=parsed["actor"],
            detail={
                "plan_code": parsed["plan_code"],
                "session_id": parsed.get("session_id"),
                "source": "stripe_webhook",
            },
        )
```

```
        await session.commit()
    except Exception:
        pass

    return {
        "received": True,
        "handled": True,
        "actor": parsed["actor"],
        "plan_code": parsed["plan_code"],
        "subscription_id": sub.id,
    }

@router.post("/webhook/stripe")
async def stripe_webhook(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Stripe webhook receiver (public path — signature verified)."""
    return await _handle_stripe_webhook(request, session)

@router.post("/webhook")
async def billing_webhook(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Stripe-compatible webhook alias (POST /api/v1/billing/webhook)."""
    return await _handle_stripe_webhook(request, session)

@router.get("/invoices")
@require_permission(Permission.TASK_READ)
async def list_invoices(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    actor = _actor(request)
    svc = get_billing_service()
    invs = await svc.list_invoices(session, actor)
    return {
        "items": [
            {
                "id": i.id,
                "period": i.period,
                "amount_cents": i.amount_cents,
                "status": i.status,
                "line_items": i.line_items,
                "issued_at": i.issued_at.isoformat() if i.issued_at else None,
            }
            for i in invs
        ],
        "total": len(invs),
    }

@router.post("/invoices/draft")
@require_permission(Permission.SYSTEM_CONFIG)
async def create_draft_invoice(
```

```
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    actor = _actor(request)
    svc = get_billing_service()
    inv = await svc.draft_invoice(session, actor)
    await session.commit()
    return {
        "invoice": {
            "id": inv.id,
            "period": inv.period,
            "amount_cents": inv.amount_cents,
            "status": inv.status,
            "line_items": inv.line_items,
        }
    }

@router.post("/quota/rollover")
@require_permission(Permission.SYSTEM_CONFIG)
async def rollover_quota(
    request: Request,
    session: AsyncSession = Depends(get_db),
    all_actors: bool = Query(False, description="Reset all subscribed actors"),
) -> dict[str, Any]:
    """Create a fresh quota balance for the current calendar month.

    Idempotent for the current period. Used by ops cron or manual admin.
    """
    svc = get_billing_service()
    await svc.ensure_default_plans(session)
    if all_actors:
        n = await svc.rollover_all_active(session)
        await session.commit()
        return {"rolled": n, "scope": "all_active"}
    actor = _actor(request)
    bal = await svc.rollover_period(session, actor)
    await session.commit()
    return {
        "rolled": 1,
        "scope": "self",
        "quota": {
            "actor": bal.actor,
            "period": bal.period,
            "task_used": bal.task_used,
            "task_limit": bal.task_limit,
            "token_used": bal.token_used,
            "token_limit": bal.token_limit,
        },
    }

# === File: backend/app/api/v1/endpoints/dashboard.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Dashboard stats API with short-TTL cache + Intelligence Center overview."""

from __future__ import annotations
```

```
from typing import Any

from fastapi import APIRouter, Depends, Query, Request
from sqlalchemy.ext.asyncio import AsyncSession

from app.core.dependencies import get_db
from app.core.rbac import Permission, get_request_role, require_permission

router = APIRouter()

@router.get("/stats")
@require_permission(Permission.TASK_READ)
async def dashboard_stats(
    request: Request,
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Return aggregated task counts for the current actor (cached 1 min)."""
    from app.core.cache.services import get_cached_dashboard

    actor = getattr(request.state, "actor", None) or "anonymous"
    role = get_request_role(request).value
    return await get_cached_dashboard(session, actor=actor, role=role)

def _build_topology(
    *,
    running: int,
    completed: int,
    failed: int,
    avg_score: float | None,
    latency_ms: float | None,
    tokens: int,
    success_rate: float | None,
    model_hint: str | None = None,
) -> dict[str, Any]:
    """Horizontal ReAct pipeline topology for ReactFlow cockpit."""
    tool_status = (
        "error" if failed > max(completed, 1) else ("warn" if failed else "ok")
    )
    planner_status = "ok" if running or completed else "idle"
    if running:
        planner_status = "ok"
    judge_status = "ok" if completed else ("warn" if failed else "idle")
    observe_status = "warn" if failed else "ok"

    lat_label = f"{round(latency_ms)} ms" if latency_ms is not None else "—"
    score_label = f"{avg_score:.1f}" if avg_score is not None else "—"
    sr = f"{success_rate * 100:.1f}%" if success_rate is not None else "—"

    nodes = [
        {
            "id": "user",
            "label": "User Request",
            "status": "ok",
            "kind": "ingress",
            "meta": {
                "type": "ingress",
                "input": "Business query / test suite",
                "output": "→ Planner",
```

```
            "latency": "< 1 ms",
            "model": "—",
        },
    },
    {
        "id": "planner",
        "label": "Planner Agent",
        "status": planner_status,
        "kind": "agent",
        "meta": {
            "type": "agent",
            "running": running,
            "input": "User query + system prompt",
            "output": "Thought / plan / tool plan",
            "token": tokens // max(running + completed, 1) if tokens else 0,
            "latency": lat_label,
            "model": model_hint or "openai-compatible",
        },
    },
    {
        "id": "tool",
        "label": "Tool Calling",
        "status": tool_status,
        "kind": "tool",
        "meta": {
            "type": "tool",
            "failed": failed,
            "input": "Function call args",
            "output": "Tool observation payload",
            "latency": "sandbox",
            "error": f"{failed} failed tasks" if failed else "",
        },
    },
    {
        "id": "observe",
        "label": "Observation",
        "status": observe_status,
        "kind": "observe",
        "meta": {
            "type": "observe",
            "input": "Tool result",
            "output": "Normalized observation",
            "latency": lat_label,
        },
    },
    {
        "id": "judge",
        "label": "LLM Judge",
        "status": judge_status,
        "kind": "judge",
```

```
            "meta": {
                "type": "judge",
                "score": avg_score,
                "input": "Trace + expected",
                "output": f"Score {score_label} · SR {sr}",
                "latency": lat_label,
                "model": "judge-engine",
            },
        },
    ]

    edges = [
        {"source": "user", "target": "planner", "label": "dispatch"},
        {"source": "planner", "target": "tool", "label": "act"},
        {"source": "tool", "target": "observe", "label": "result"},
        {"source": "observe", "target": "judge", "label": "score"},
    ]
    if failed:
        edges.append(
            {
                "source": "observe",
                "target": "planner",
                "type": "loop",
                "label": "retry",
            }
        )

    return {
        "layout": "horizontal",
        "nodes": nodes,
        "edges": edges,
        "legend": [
            {"status": "ok", "label": "Healthy"},
            {"status": "warn", "label": "Degraded"},
            {"status": "error", "label": "Failure"},
            {"status": "idle", "label": "Idle"},
        ],
    }

@router.get("")
@router.get("/overview")
@require_permission(Permission.TASK_READ)
async def dashboard_overview(
    request: Request,
    days: int = Query(7, ge=1, le=90),
    session: AsyncSession = Depends(get_db),
) -> dict[str, Any]:
    """Intelligence Center cockpit payload for ECharts + ReactFlow."""
    from app.core.cache.services import get_cached_dashboard
    from app.core.observability.business_kpis import compute_kpis
    from app.core.observability.timeseries import compute_dashboard_series

    actor = getattr(request.state, "actor", None) or "anonymous"
    role = get_request_role(request).value
# === SEGMENT_BREAK ===
```

— 第 30 页之后省略中间部分源代码 —

（前30页完，以下为后30页）

```
class ExperimentRun(PKMixin, TenantMixin, TimestampMixin, Base):
    """One evaluation run inside an experiment (maps 1:1 to a Task)."""

    __tablename__ = "experiment_runs"
    __table_args__ = (
        UniqueConstraint("experiment_id", "label", name="uq_experiment_run_label"),
        Index("ix_experiment_runs_tenant", "tenant_id"),
    )

    experiment_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("experiments.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
        comment="所属实验",
    )
    task_id: Mapped[str] = mapped_column(
        String(36),
        ForeignKey("tasks.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
        comment="对应评测任务",
    )
    label: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
        comment="变体标签，如 gpt-4o / http-agent-v1",
    )
    agent_config: Mapped[dict[str, Any]] = mapped_column(
        JSON,
        nullable=False,
        default=dict,
        comment="该 run 使用的 agent_config 快照",
    )

    experiment: Mapped[Experiment] = relationship(back_populates="runs")
    task: Mapped[Task] = relationship()

    def __repr__(self) -> str:
        return f"<ExperimentRun id={self.id} label={self.label!r} task={self.task_id}>"

# === File: backend/app/models/media_asset.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""MediaAsset — uploaded multimodal files linked to tasks/suites."""

from __future__ import annotations

from typing import TYPE_CHECKING, Any

from sqlalchemy import ForeignKey, Index, Integer, JSON, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
    from app.models.task import Task
    from app.models.test_suite import TestSuite

class MediaAsset(PKMixin, TenantMixin, TimestampMixin, Base):
    """Stored multimodal artifact with extraction results."""

    __tablename__ = "media_assets"
    __table_args__ = (
```

```
        Index("ix_media_assets_task_id", "task_id"),
        Index("ix_media_assets_suite_id", "test_suite_id"),
        Index("ix_media_assets_created_by", "created_by"),
        Index("ix_media_assets_kind", "media_kind"),
        Index("ix_media_assets_tenant", "tenant_id"),
    )

    filename: Mapped[str] = mapped_column(String(512), nullable=False)
    content_type: Mapped[str] = mapped_column(
        String(128), nullable=False, default="application/octet-stream"
    )
    size_bytes: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    sha256: Mapped[str] = mapped_column(String(64), nullable=False, default="")
    media_kind: Mapped[str] = mapped_column(String(32), nullable=False, default="other")

    storage_backend: Mapped[str] = mapped_column(
        String(32), nullable=False, default="local"
    )
    storage_key: Mapped[str] = mapped_column(String(1024), nullable=False)

    extracted_text: Mapped[str] = mapped_column(Text, nullable=False, default="")
    features: Mapped[dict[str, Any] | None] = mapped_column(
        JSON, nullable=True, default=None
    )
    extract_meta: Mapped[dict[str, Any] | None] = mapped_column(
        JSON, nullable=True, default=None
    )

    task_id: Mapped[str | None] = mapped_column(
        String(36),
        ForeignKey("tasks.id", ondelete="SET NULL"),
        nullable=True,
    )
    test_suite_id: Mapped[str | None] = mapped_column(
        String(36),
        ForeignKey("test_suites.id", ondelete="SET NULL"),
        nullable=True,
    )
    created_by: Mapped[str] = mapped_column(
        String(100), nullable=False, default="anonymous"
    )

    task: Mapped[Task | None] = relationship()
    test_suite: Mapped[TestSuite | None] = relationship()

    def __repr__(self) -> str:
        return (
            f"<MediaAsset id={self.id} kind={self.media_kind!r} file={self.filename!r}>"
        )

# === File: backend/app/models/metric_score.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""MetricScore model —— 单项评分指标记录。"""

from typing import TYPE_CHECKING

from sqlalchemy import Boolean, Float, ForeignKey, Index, JSON, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
```

```
    from app.models.trace import Trace

class MetricScore(PKMixin, TenantMixin, TimestampMixin, Base):
    """评测指标得分。

    记录每个 Trace 在某个指标维度（如准确率、工具调用正确性等）的评分，
    支持 LLM 自动评分与人工审核覆盖。
    """

    __tablename__ = "metric_scores"
    __table_args__ = (
        Index("ix_metric_scores_trace_metric", "trace_id", "metric_name"),
        Index("ix_metric_scores_human_reviewed", "is_human_reviewed"),
        Index("ix_metric_scores_tenant", "tenant_id"),
    )

    trace_id: Mapped[str] = mapped_column(
        ForeignKey("traces.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
        comment="FK to trace",
    )
    metric_name: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
        comment="metric name",
    )
    score: Mapped[float] = mapped_column(
        Float,
        nullable=False,
        comment="score 0-100",
    )
    reason: Mapped[str] = mapped_column(
        Text,
        nullable=False,
        default="",
        comment="deduction reason",
    )
    extra_metadata: Mapped[dict | None] = mapped_column(
        JSON,
        nullable=True,
        default=None,
        comment="extra metadata",
    )

    confidence: Mapped[float | None] = mapped_column(
        Float,
        nullable=True,
        default=None,
        comment="LLM 评分置信度 0.0 ~ 1.0",
    )

    is_human_reviewed: Mapped[bool] = mapped_column(
        Boolean,
        nullable=False,
        default=False,
```

```
        comment="是否经过人工审核",
    )
    human_score: Mapped[float | None] = mapped_column(
        Float,
        nullable=True,
        default=None,
        comment="人工审核给出的分数（覆盖 LLM 评分）",
    )
    reviewer: Mapped[str | None] = mapped_column(
        String(100),
        nullable=True,
        default=None,
        comment="人工审核人员标识",
    )

    trace: Mapped["Trace"] = relationship(back_populates="metric_scores")

    @property
    def effective_score(self) -> float:
        """返回最终有效分数：优先使用人工审核分数。"""
        if self.is_human_reviewed and self.human_score is not None:
            return self.human_score
        return self.score

    def __repr__(self) -> str:
        return f"<MetricScore name={self.metric_name!r} score={self.score} reviewed={self.is_human_reviewed}>"

# === File: backend/app/models/slow_task.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Persisted slow-task samples for diagnostics across restarts."""

from __future__ import annotations

from datetime import datetime

from sqlalchemy import DateTime, Float, Index, JSON, String, func
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, PKMixin

class SlowTaskEvent(PKMixin, Base):
    """One slow agent/judge/pipeline sample above configured threshold."""

    __tablename__ = "slow_task_events"
    __table_args__ = (
        Index("ix_slow_task_created", "created_at"),
        Index("ix_slow_task_stage_created", "stage", "created_at"),
        Index("ix_slow_task_trace", "trace_id"),
        Index("ix_slow_task_actor", "actor"),
    )

    stage: Mapped[str] = mapped_column(String(32), nullable=False, index=True)
    duration_sec: Mapped[float] = mapped_column(Float, nullable=False)
    threshold_sec: Mapped[float] = mapped_column(Float, nullable=False)
    ref_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
    status: Mapped[str] = mapped_column(String(32), nullable=False, default="ok")
    trace_id: Mapped[str | None] = mapped_column(String(64), nullable=True)
    actor: Mapped[str | None] = mapped_column(String(100), nullable=True)
    extra: Mapped[dict] = mapped_column(JSON, nullable=False, default=dict)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
```

```
        nullable=False,
    )

# === File: backend/app/models/task.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Task 任务模型 —— 表示一次评测任务的顶层聚合根。"""

import enum
from typing import TYPE_CHECKING

from sqlalchemy import Boolean, Index, JSON, Enum, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
    from app.models.test_suite import TestSuite

class TaskStatus(str, enum.Enum):
    """任务状态枚举 —— 生产级状态机。

    状态流转:
        CREATED ──► QUEUED ──► RUNNING ──► JUDGING ──► COMPLETED
                                  │           │
                                  ▼           ▼
                               FAILED      FAILED
                                  │
                                  ▼
                              CANCELLED / TIMEOUT
    """

    CREATED = "created"
    QUEUED = "queued"
    RUNNING = "running"
    WAITING_TOOL = "waiting_tool"
    JUDGING = "judging"
    COMPLETED = "completed"
    FAILED = "failed"
    CANCELLED = "cancelled"
    TIMEOUT = "timeout"

    @classmethod
    def allowed_transitions(cls) -> dict["TaskStatus", set["TaskStatus"]]:
        """返回每个状态允许流转的目标状态集合。"""
        return {
            cls.CREATED: {cls.QUEUED, cls.CANCELLED},
            cls.QUEUED: {cls.RUNNING, cls.CANCELLED, cls.TIMEOUT},
            cls.RUNNING: {
                cls.WAITING_TOOL,
                cls.JUDGING,
                cls.FAILED,
                cls.CANCELLED,
                cls.TIMEOUT,
            },
            cls.WAITING_TOOL: {cls.RUNNING, cls.FAILED, cls.CANCELLED, cls.TIMEOUT},
            cls.JUDGING: {cls.COMPLETED, cls.FAILED, cls.CANCELLED},
            cls.COMPLETED: set(),
            cls.FAILED: set(),
            cls.CANCELLED: set(),
            cls.TIMEOUT: set(),
```

```
        }

    def can_transition_to(self, target: "TaskStatus") -> bool:
        """检查是否允许从当前状态流转到目标状态。"""
        return target in self.allowed_transitions().get(self, set())

class Task(PKMixin, TenantMixin, TimestampMixin, Base):
    """评测任务。

    一个 Task 包含多个 TestSuite 测试用例，执行时逐条运行并汇总评分。
    """

    __tablename__ = "tasks"
    __table_args__ = (
        Index(
            "ix_tasks_owner_archived_created",
            "created_by",
            "is_archived",
            "created_at",
        ),
        Index("ix_tasks_status_created", "status", "created_at"),
        Index("ix_tasks_created_by", "created_by"),
        Index("ix_tasks_celery_task_id", "celery_task_id"),
        Index("ix_tasks_is_archived", "is_archived"),
        Index("ix_tasks_tenant_created", "tenant_id", "created_at"),
    )

    name: Mapped[str] = mapped_column(
        String(255),
        nullable=False,
        comment="任务名称",
    )
    description: Mapped[str] = mapped_column(
        Text,
        nullable=False,
        default="",
        comment="任务描述",
    )
    status: Mapped[TaskStatus] = mapped_column(
        Enum(
            TaskStatus,
            name="task_status",
            native_enum=False,
            length=20,
            values_callable=lambda enum_cls: [e.value for e in enum_cls],
        ),
        nullable=False,
        default=TaskStatus.CREATED,
        comment="任务状态",
    )
    agent_config: Mapped[dict] = mapped_column(
        JSON,
        nullable=False,
        default=dict,
        comment="Agent 配置参数（JSON 对象）",
```

```
    )
    celery_task_id: Mapped[str | None] = mapped_column(
        String(255),
        nullable=True,
        default=None,
        comment="Celery 异步任务 ID，用于取消和状态追踪",
    )
    is_archived: Mapped[bool] = mapped_column(
        Boolean,
        nullable=False,
        default=False,
        comment="是否已归档（软归档，列表默认隐藏）",
    )
    created_by: Mapped[str] = mapped_column(
        String(100),
        nullable=False,
        default="anonymous",
        comment="创建者 actor（API Key 映射名），用于轻量多租户隔离",
    )

    test_suites: Mapped[list["TestSuite"]] = relationship(
        back_populates="task",
        cascade="all, delete-orphan",
    )

    def __repr__(self) -> str:
        return f"<Task id={self.id} name={self.name!r} status={self.status.value}>"

# === File: backend/app/models/tenant.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Enterprise multi-tenant models: tenants + membership."""

from __future__ import annotations

from sqlalchemy import Index, String, UniqueConstraint
from sqlalchemy.orm import Mapped, mapped_column

from app.models.base import Base, PKMixin, TimestampMixin

class Tenant(PKMixin, TimestampMixin, Base):
    """Organization / workspace boundary for SaaS and private multi-team deploys."""

    __tablename__ = "tenants"
    __table_args__ = (
        UniqueConstraint("slug", name="uq_tenants_slug"),
        Index("ix_tenants_status", "status"),
    )

    name: Mapped[str] = mapped_column(String(200), nullable=False)
    slug: Mapped[str] = mapped_column(String(100), nullable=False, index=True)
    status: Mapped[str] = mapped_column(
        String(32), nullable=False, default="active"
    )  # active|suspended|deleted
    plan_id: Mapped[str | None] = mapped_column(String(36), nullable=True, index=True)

    def __repr__(self) -> str:
        return f"<Tenant {self.slug} status={self.status}>"

class TenantMember(PKMixin, TimestampMixin, Base):
    """Actor membership inside a tenant (role is tenant-scoped RBAC)."""

    __tablename__ = "tenant_members"
    __table_args__ = (
```

```
        UniqueConstraint("tenant_id", "user_id", name="uq_tenant_member"),
        Index("ix_tenant_members_user", "user_id"),
        Index("ix_tenant_members_tenant_role", "tenant_id", "role"),
    )

    tenant_id: Mapped[str] = mapped_column(String(36), nullable=False, index=True)
    user_id: Mapped[str] = mapped_column(String(100), nullable=False)
    role: Mapped[str] = mapped_column(
        String(32),
        nullable=False,
        default="member",
    )  # tenant_admin|manager|reviewer|member|viewer

    def __repr__(self) -> str:
        return f"<TenantMember {self.user_id}@{self.tenant_id} role={self.role}>"

# === File: backend/app/models/test_suite.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""TestSuite model."""

from typing import TYPE_CHECKING
from sqlalchemy import ForeignKey, JSON, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
    from app.models.task import Task
    from app.models.trace import Trace

class TestSuite(PKMixin, TenantMixin, TimestampMixin, Base):
    __tablename__ = "test_suites"

    task_id: Mapped[str] = mapped_column(
        ForeignKey("tasks.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
        comment="FK to task",
    )
    user_query: Mapped[str] = mapped_column(
        Text, nullable=False, comment="user instruction"
    )
    expected_output: Mapped[str] = mapped_column(
        Text, nullable=False, default="", comment="expected output"
    )
    expected_tools: Mapped[list] = mapped_column(
        JSON, nullable=False, default=list, comment="expected tool names"
    )
    extra_metadata: Mapped[dict | None] = mapped_column(
        JSON, nullable=True, default=None, comment="extra metadata"
    )

    task: Mapped["Task"] = relationship(back_populates="test_suites")
    traces: Mapped[list["Trace"]] = relationship(
        back_populates="test_suite", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"<TestSuite id={self.id}>"

# === File: backend/app/models/trace.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Trace 模型 —— 记录 Agent 执行一次测试用例的完整轨迹。"""
```

```
import enum
from typing import TYPE_CHECKING

from sqlalchemy import Enum, Float, ForeignKey, Index, Integer, JSON, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.models.base import Base, PKMixin, TenantMixin, TimestampMixin

if TYPE_CHECKING:
    from app.models.metric_score import MetricScore
    from app.models.test_suite import TestSuite

class TraceStatus(str, enum.Enum):
    """轨迹执行状态。"""

    SUCCESS = "success"
    FAILED = "failed"

class Trace(PKMixin, TenantMixin, TimestampMixin, Base):
    """执行轨迹。

    保存 Agent 运行一次测试用例的完整 ReAct 步骤、Token 消耗和响应时间，
    是评分和链路可视化的核心数据源。
    """

    __tablename__ = "traces"
    __table_args__ = (
        Index("ix_traces_suite_created", "test_suite_id", "created_at"),
        Index("ix_traces_created_at", "created_at"),
        Index("ix_traces_tenant_created", "tenant_id", "created_at"),
    )

    test_suite_id: Mapped[str] = mapped_column(
        ForeignKey("test_suites.id", ondelete="CASCADE"),
        nullable=False,
        index=True,
        comment="所属测试用例 ID",
    )
    user_query: Mapped[str] = mapped_column(
        Text,
        nullable=False,
        comment="本次执行使用的用户指令（快照）",
    )
    steps: Mapped[list] = mapped_column(
        JSON,
        nullable=False,
        default=list,
        comment="ReAct 步骤数组，每步包含 thought/action/observation 等字段",
    )
    total_tokens: Mapped[int] = mapped_column(
        Integer,
        nullable=False,
        default=0,
        comment="本次执行消耗的总 Token 数",
    )
    response_time_ms: Mapped[int] = mapped_column(
        Integer,
        nullable=False,
        default=0,
```

```
        comment="执行耗时（毫秒）",
    )
    status: Mapped[TraceStatus] = mapped_column(
        Enum(
            TraceStatus,
            name="trace_status",
            values_callable=lambda enum_cls: [e.value for e in enum_cls],
        ),
        nullable=False,
        default=TraceStatus.SUCCESS,
        comment="执行状态：success / failed",
    )

    prompt_tokens: Mapped[int] = mapped_column(
        Integer,
        nullable=False,
        default=0,
        comment="输入 Prompt Token 数",
    )
    completion_tokens: Mapped[int] = mapped_column(
        Integer,
        nullable=False,
        default=0,
        comment="输出 Completion Token 数",
    )
    cost: Mapped[float] = mapped_column(
        Float,
        nullable=False,
        default=0.0,
        comment="本次执行费用（USD）",
    )

    agent_version: Mapped[str | None] = mapped_column(
        String(100),
        nullable=True,
        default=None,
        comment="Agent 实现版本号",
    )
    prompt_version: Mapped[str | None] = mapped_column(
        String(100),
        nullable=True,
        default=None,
        comment="Prompt 模板版本号",
    )
    model_version: Mapped[str | None] = mapped_column(
        String(100),
        nullable=True,
        default=None,
        comment="LLM 模型版本（如 gpt-4o-2024-05-13）",
    )
    tool_version: Mapped[str | None] = mapped_column(
        String(100),
```

```
        nullable=True,
        default=None,
        comment="工具集版本号",
    )

    test_suite: Mapped["TestSuite"] = relationship(back_populates="traces")
    metric_scores: Mapped[list["MetricScore"]] = relationship(
        back_populates="trace",
        cascade="all, delete-orphan",
    )

    def __repr__(self) -> str:
        return f"<Trace id={self.id} status={self.status.value} tokens={self.total_tokens}>"

# === File: backend/app/plugins/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Built-in / example plugins package (optional load via PLUGIN_MODULES)."""

# === File: backend/app/plugins/examples/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Example plugins demonstrating AgentRunner / Judge / Tool / Hook types."""

# === File: backend/app/plugins/examples/audit_hooks.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Example HOOK plugin — records pre/post pipeline events in-memory."""

from __future__ import annotations

from typing import Any

from app.core.plugins.base import (
    HOOK_POST_AGENT_RUN,
    HOOK_POST_JUDGE,
    HOOK_PRE_AGENT_RUN,
    HOOK_PRE_JUDGE,
    BasePlugin,
    PluginContext,
    PluginMeta,
    PluginType,
)

_EVENT_LOG: list[dict[str, Any]] = []
_MAX = 200

def get_event_log() -> list[dict[str, Any]]:
    return list(_EVENT_LOG)

def clear_event_log() -> None:
    _EVENT_LOG.clear()

def _append(event: str, payload: dict[str, Any]) -> dict[str, Any]:
    item = {"event": event, **{k: v for k, v in payload.items() if k != "event"}}
    _EVENT_LOG.append(item)
    if len(_EVENT_LOG) > _MAX:
        del _EVENT_LOG[: len(_EVENT_LOG) - _MAX]
    return item

class Plugin(BasePlugin):
    meta = PluginMeta(
        name="audit_hooks",
        version="1.0.0",
        plugin_type=PluginType.HOOK,
        description="Logs pre/post agent and judge hooks (example).",
        author="AgentFlow-Eval",
        provides=[],
    )
```

```
    def on_activate(self, ctx: PluginContext) -> None:
        assert ctx.register_hook is not None

        def pre_agent(payload: dict[str, Any]) -> dict[str, Any]:
            return _append(HOOK_PRE_AGENT_RUN, payload)

        def post_agent(payload: dict[str, Any]) -> dict[str, Any]:
            return _append(HOOK_POST_AGENT_RUN, payload)

        def pre_judge(payload: dict[str, Any]) -> dict[str, Any]:
            return _append(HOOK_PRE_JUDGE, payload)

        def post_judge(payload: dict[str, Any]) -> dict[str, Any]:
            return _append(HOOK_POST_JUDGE, payload)

        ctx.register_hook(HOOK_PRE_AGENT_RUN, pre_agent, priority=50)
        ctx.register_hook(HOOK_POST_AGENT_RUN, post_agent, priority=50)
        ctx.register_hook(HOOK_PRE_JUDGE, pre_judge, priority=50)
        ctx.register_hook(HOOK_POST_JUDGE, post_judge, priority=50)

# === File: backend/app/plugins/examples/echo_runner.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Example AGENT_RUNNER plugin — deterministic echo runner."""

from __future__ import annotations

from datetime import datetime, timezone
from typing import Any

from app.core.agent_runner.base import AgentResult, BaseAgentRunner
from app.core.plugins.base import BasePlugin, PluginContext, PluginMeta, PluginType

class EchoAgentRunner(BaseAgentRunner):
    """Returns the user query as a single final answer step (no LLM)."""

    async def run(
        self,
        query: str,
        tools: list[Any] | None = None,
        *,
        agent_config: dict[str, Any] | None = None,
    ) -> AgentResult:
        _ = tools
        cfg = dict(agent_config or {})
        prefix = str(cfg.get("prefix") or "ECHO: ")
        text = f"{prefix}{query}"
        steps = [
            {
                "type": "thought",
                "content": "Echo runner reflects the query without external calls.",
            },
            {
                "type": "final",
                "content": text,
                "answer": text,
            },
        ]
        return AgentResult(
            steps=steps,
            total_tokens=0,
            response_time_ms=1,
            status="success",
```

```
            final_answer=text,
            runner="echo",
            finished_at=datetime.now(timezone.utc),
        )

class Plugin(BasePlugin):
    meta = PluginMeta(
        name="echo_runner",
        version="1.0.0",
        plugin_type=PluginType.AGENT_RUNNER,
        description="Deterministic echo AgentRunner (example).",
        author="AgentFlow-Eval",
        provides=["echo", "echo_runner"],
    )

    def on_activate(self, ctx: PluginContext) -> None:
        assert ctx.register_runner is not None

        def factory(agent_config: dict[str, Any]) -> EchoAgentRunner:
            return EchoAgentRunner()

        ctx.register_runner("echo", factory)
        ctx.register_runner("echo_runner", factory)

# === File: backend/app/plugins/examples/echo_tool.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Example TOOL plugin — echo arguments as JSON."""

from __future__ import annotations

import json
from typing import Any

from app.core.plugins.base import BasePlugin, PluginContext, PluginMeta, PluginType

def _echo_tool(**kwargs: Any) -> str:
    return json.dumps({"echo": kwargs}, ensure_ascii=False)

class Plugin(BasePlugin):
    meta = PluginMeta(
        name="echo_tool",
        version="1.0.0",
        plugin_type=PluginType.TOOL,
        description="Echo tool arguments as JSON (example).",
        author="AgentFlow-Eval",
        provides=["echo"],
    )

    def on_activate(self, ctx: PluginContext) -> None:
        assert ctx.register_tool is not None
        ctx.register_tool(
            "echo",
            _echo_tool,
            description="Echo back all tool arguments as a JSON object.",
            parameters={
                "message": {
                    "type": "string",
                    "description": "Optional message to echo",
                },
            },
            required=[],
            network=False,
```

```
        )

# === File: backend/app/plugins/examples/length_judge.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Example JUDGE plugin — score based on answer length similarity."""

from __future__ import annotations

from typing import Any

from app.core.judge_engine.base import BaseJudge, JudgeResult
from app.core.plugins.base import BasePlugin, PluginContext, PluginMeta, PluginType

class LengthJudge(BaseJudge):
    """Simple deterministic judge for demos and tests."""

    async def evaluate(
        self,
        trace_steps: list[dict[str, Any]],
        expected_output: str,
        expected_tools: list[str],
    ) -> JudgeResult:
        answer = ""
        for step in reversed(trace_steps or []):
            if not isinstance(step, dict):
                continue
            for key in ("content", "output", "answer", "observation"):
                if step.get(key):
                    answer = str(step[key])
                    break
            if answer:
                break

        exp = (expected_output or "").strip()
        ans = answer.strip()
        if not exp and not ans:
            score = 1.0
        elif not exp or not ans:
            score = 0.0
        else:
            ratio = min(len(ans), len(exp)) / max(len(ans), len(exp))
            score = round(float(ratio), 4)

        tools_used = []
        for step in trace_steps or []:
            if isinstance(step, dict) and step.get("tool"):
                tools_used.append(str(step["tool"]))
            if isinstance(step, dict) and step.get("tool_name"):
                tools_used.append(str(step["tool_name"]))

        tool_score = 1.0
        if expected_tools:
            hit = sum(1 for t in expected_tools if t in tools_used)
            tool_score = hit / len(expected_tools)

        total = round(0.7 * score + 0.3 * tool_score, 4)
        return JudgeResult(
            scores={
                "length_similarity": score,
                "tool_coverage": tool_score,
            },
```

```
            total=total,
            reason=f"length_ratio={score}, tool_coverage={tool_score}",
            token_cost=0,
        )

class Plugin(BasePlugin):
    meta = PluginMeta(
        name="length_judge",
        version="1.0.0",
        plugin_type=PluginType.JUDGE,
        description="Rule judge: answer length ratio + tool coverage.",
        author="AgentFlow-Eval",
        provides=["length", "length_judge"],
    )

    def on_activate(self, ctx: PluginContext) -> None:
        assert ctx.register_judge is not None

        def factory() -> LengthJudge:
            return LengthJudge()

        ctx.register_judge("length", factory)
        ctx.register_judge("length_judge", factory)

# === File: backend/app/schemas/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Pydantic 请求/响应模型包。"""

from app.schemas.task import (
    TaskCreate,
    TaskResponse,
    TaskListResponse,
    TaskStatusUpdate,
)
from app.schemas.trace import (
    TraceResponse,
    TraceListResponse,
    MetricScoreResponse,
    JudgeResultResponse,
)

__all__ = [
    "TaskCreate",
    "TaskResponse",
    "TaskListResponse",
    "TaskStatusUpdate",
    "TraceResponse",
    "TraceListResponse",
    "MetricScoreResponse",
    "JudgeResultResponse",
]

# === File: backend/app/schemas/ab_test.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Schemas for online A/B testing APIs."""

from __future__ import annotations

from datetime import datetime
from typing import Any

from pydantic import BaseModel, Field, field_validator

class ABVariantCreate(BaseModel):
```

```
    key: str = Field(..., min_length=1, max_length=100)
    name: str = Field(default="")
    weight: float = Field(default=1.0, gt=0)
    is_control: bool = False
    payload: dict[str, Any] = Field(default_factory=dict)
    description: str = ""

class ABExperimentCreate(BaseModel):
    key: str = Field(..., min_length=1, max_length=100, pattern=r"^[a-zA-Z0-9_\-]+$")
    name: str = Field(..., min_length=1, max_length=255)
    description: str = ""
    primary_metric: str = Field(default="conversion")
    alpha: float = Field(default=0.05, gt=0, lt=1)
    min_sample_size: int = Field(default=100, ge=1)
    variants: list[ABVariantCreate] = Field(..., min_length=2)
    source_experiment_id: str | None = None
    config: dict[str, Any] = Field(default_factory=dict)
    start_immediately: bool = False

    @field_validator("variants")
    @classmethod
    def validate_variants(cls, v: list[ABVariantCreate]) -> list[ABVariantCreate]:
        keys = [x.key for x in v]
        if len(keys) != len(set(keys)):
            raise ValueError("variant keys must be unique")
        controls = [x for x in v if x.is_control]
        if len(controls) > 1:
            raise ValueError("at most one control variant")
        return v

class ABVariantResponse(BaseModel):
    id: str
    key: str
    name: str
    weight: float
    is_control: bool
    payload: dict[str, Any] = Field(default_factory=dict)
    description: str = ""

    model_config = {"from_attributes": True}

class ABExperimentResponse(BaseModel):
    id: str
    key: str
    name: str
    description: str
    status: str
    alpha: float
    min_sample_size: int
    primary_metric: str
    control_variant_key: str | None = None
    source_experiment_id: str | None = None
    config: dict[str, Any] = Field(default_factory=dict)
    created_by: str = "anonymous"
    started_at: datetime | None = None
```

```
    ended_at: datetime | None = None
    winner_variant_key: str | None = None
    created_at: datetime | None = None
    variants: list[ABVariantResponse] = Field(default_factory=list)

    model_config = {"from_attributes": True}

class ABExperimentListResponse(BaseModel):
    items: list[ABExperimentResponse]
    total: int
    page: int = 1
    page_size: int = 20

class ABAssignRequest(BaseModel):
    unit_id: str = Field(..., min_length=1, max_length=128)
    context: dict[str, Any] | None = None
    record_exposure: bool = True

class ABAssignResponse(BaseModel):
    experiment_id: str
    experiment_key: str
    unit_id: str
    variant_key: str
    bucket: int
    is_new: bool
    is_control: bool
    payload: dict[str, Any] = Field(default_factory=dict)
    status: str

class ABTrackRequest(BaseModel):
    unit_id: str = Field(..., min_length=1, max_length=128)
    event_type: str = Field(..., description="exposure | conversion | metric")
    metric_name: str | None = None
    metric_value: float | None = None
    properties: dict[str, Any] | None = None
    auto_assign: bool = True

class ABSampleSizeRequest(BaseModel):
    baseline_rate: float = Field(..., ge=0, le=1)
    mde: float = Field(
        ..., gt=0, le=1, description="Absolute minimum detectable effect"
    )
    alpha: float = Field(default=0.05, gt=0, lt=1)
    power: float = Field(default=0.8, gt=0, lt=1)

class ABStatusUpdate(BaseModel):
    status: str = Field(..., pattern=r"^(draft|running|paused|completed|archived)$")

# === File: backend/app/schemas/experiment.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Pydantic schemas for Experiment comparison APIs."""

from __future__ import annotations

from datetime import datetime
from typing import Any

from pydantic import BaseModel, Field, field_validator

class SuiteCase(BaseModel):
    """Single test case in an experiment suite snapshot."""

    user_query: str = Field(..., min_length=1)
    expected_output: str = Field(default="")
```

```
    expected_tools: list[str] = Field(default_factory=list)

class ExperimentVariant(BaseModel):
    """One agent configuration variant to compare."""

    label: str = Field(..., min_length=1, max_length=100, description="变体标签")
    agent_config: dict[str, Any] = Field(
        default_factory=dict,
        description="Agent 配置（runner/model/endpoint_url 等）",
    )

class ExperimentCreate(BaseModel):
    """Create experiment from base task and/or explicit suites + variants."""

    name: str = Field(..., min_length=1, max_length=255)
    description: str = Field(default="")
    base_task_id: str | None = Field(
        default=None,
        description="从已有任务复制测试套件（与 suites 二选一或合并）",
    )
    suites: list[SuiteCase] = Field(
        default_factory=list,
        description="显式用例列表；可与 base_task_id 并用",
    )
    variants: list[ExperimentVariant] = Field(
        ...,
        min_length=1,
        description="至少一个变体；创建后会为每个变体生成 Task 并排队执行",
    )
    auto_execute: bool = Field(
        default=True,
        description="是否立即排队执行各变体任务",
    )

    @field_validator("variants")
    @classmethod
    def unique_labels(cls, v: list[ExperimentVariant]) -> list[ExperimentVariant]:
        labels = [x.label for x in v]
        if len(labels) != len(set(labels)):
            raise ValueError("variant labels must be unique within an experiment")
        return v

class ExperimentRunResponse(BaseModel):
    """Single run in list/detail responses."""

    id: str
    experiment_id: str
    task_id: str
    label: str
    agent_config: dict[str, Any]
    task_status: str | None = None
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class ExperimentResponse(BaseModel):
    """Experiment detail."""

    id: str
    name: str
```

```
    description: str
    base_task_id: str | None = None
    suite_count: int = 0
    created_by: str = "anonymous"
    created_at: datetime | None = None
    updated_at: datetime | None = None
    runs: list[ExperimentRunResponse] = Field(default_factory=list)

    model_config = {"from_attributes": True}

class ExperimentListResponse(BaseModel):
    """Paginated experiment list."""

    items: list[ExperimentResponse]
    total: int
    page: int = 1
    page_size: int = 20

class RunCompareItem(BaseModel):
    """Aggregated scores for one experiment run."""

    label: str
    task_id: str
    task_status: str
    average_score: float = 0.0
    dimension_scores: dict[str, float] = Field(default_factory=dict)
    total_tokens: int = 0
    total_time_ms: int = 0
    suite_count: int = 0
    scored_traces: int = 0

class ExperimentCompareResponse(BaseModel):
    """Side-by-side comparison of experiment runs."""

    experiment_id: str
    name: str
    suite_count: int
    runs: list[RunCompareItem]
    best_label: str | None = None
    delta_vs_best: dict[str, float] = Field(
        default_factory=dict,
        description="各变体相对最高分的总分差值（best 为 0）",
    )

# === File: backend/app/schemas/media.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Pydantic schemas for multimodal media APIs."""

from __future__ import annotations

from datetime import datetime
from typing import Any

from pydantic import BaseModel, Field

class MediaAssetResponse(BaseModel):
    id: str
    filename: str
    content_type: str
    size_bytes: int
    sha256: str
    media_kind: str
    storage_backend: str
```

```
    storage_key: str
    extracted_text: str = ""
    features: dict[str, Any] | None = None
    extract_meta: dict[str, Any] | None = None
    task_id: str | None = None
    test_suite_id: str | None = None
    created_by: str = "anonymous"
    created_at: datetime | None = None

    model_config = {"from_attributes": True}

class MediaExtractResponse(BaseModel):
    asset_id: str
    kind: str
    text: str
    features: dict[str, Any] = Field(default_factory=dict)
    metadata: dict[str, Any] = Field(default_factory=dict)
    pages: int | None = None
    tables: list[dict[str, Any]] = Field(default_factory=list)
    warnings: list[str] = Field(default_factory=list)

class MultimodalEvalRequest(BaseModel):
    """Evaluate a stored media asset (or re-score extraction)."""

    query: str = Field(default="", description="Evaluation question / prompt")
    expected_text: str = Field(default="", description="Expected content or caption")
    use_vision_llm: bool = Field(
        default=True,
        description="Use GPT-4V/gpt-4o style vision LLM when image + API key available",
    )
    model: str | None = Field(default=None, description="Override vision model")

class MultimodalEvalResponse(BaseModel):
    asset_id: str
    mode: str
    scores: dict[str, float]
    total: float
    reason: str
    kind: str
    token_cost: int = 0
    degraded: bool = False
    model: str | None = None
    extraction: MediaExtractResponse | None = None

class SupportedFormatsResponse(BaseModel):
    extensions: list[str]
    kinds: list[str]
    max_upload_bytes: int
    storage_backend: str

# === File: backend/app/schemas/task.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Task 相关的 Pydantic 请求/响应模型。"""

from datetime import datetime
from typing import Any

from pydantic import BaseModel, Field

class TaskCreate(BaseModel):
    """创建评测任务的请求体。"""
```

```
    name: str = Field(..., min_length=1, max_length=255, description="任务名称")
    description: str = Field("", description="任务描述")
    agent_config: dict[str, Any] = Field(
        default_factory=lambda: {"model": "gpt-4o", "temperature": 0},
        description="Agent 配置参数",
    )

class TaskStatusUpdate(BaseModel):
    """更新任务状态的请求体。"""

    status: str = Field(
        ...,
        pattern=r"^(created|queued|running|waiting_tool|judging|completed|failed|cancelled|timeout)$",
        description="新状态",
    )

class TaskResponse(BaseModel):
    """任务响应模型。"""

    id: str
    name: str
    description: str
    status: str
    agent_config: dict[str, Any]
    celery_task_id: str | None = None
    is_archived: bool = False
    created_by: str = "anonymous"
    created_at: datetime | None = None
    updated_at: datetime | None = None
    test_suite_count: int = 0

    model_config = {"from_attributes": True}

class TaskListResponse(BaseModel):
    """任务列表响应。"""

    items: list[TaskResponse]
    total: int
    page: int = 1
    page_size: int = 20

# === File: backend/app/schemas/trace.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Trace and MetricScore Pydantic schemas."""

from datetime import datetime
from typing import Any

from pydantic import BaseModel, Field

class MetricScoreResponse(BaseModel):
    """Metric score response schema."""

    id: str
    metric_name: str
    score: float
    reason: str
    extra_metadata: dict[str, Any] | None = None

    model_config = {"from_attributes": True}

class TokenUsage(BaseModel):
    """Token usage breakdown for observability UIs."""

    prompt_tokens: int = 0
    completion_tokens: int = 0
```

```
    total_tokens: int = 0

class TraceResponse(BaseModel):
    """Execution trace response schema."""

    id: str
    test_suite_id: str
    user_query: str
    steps: list[dict[str, Any]]
    total_tokens: int
    response_time_ms: int
    status: str
    created_at: datetime | None = None
    metric_scores: list[MetricScoreResponse] = []
    prompt_tokens: int = 0
    completion_tokens: int = 0
    cost: float = 0.0
    agent_version: str | None = None
    prompt_version: str | None = None
    model_version: str | None = None
    tool_version: str | None = None
    token_usage: TokenUsage | None = None

    model_config = {
        "from_attributes": True,
        "protected_namespaces": (),
    }

class TraceListResponse(BaseModel):
    """Trace list response schema."""

    items: list[TraceResponse]
    total: int

class JudgeResultResponse(BaseModel):
    """LLM Judge scoring result schema."""

    scores: dict[str, float] = Field(..., description="scores per dimension")
    total: float = Field(..., ge=0, le=100, description="weighted total score")
    reason: str = Field("", description="deduction reason details")
    token_cost: int = Field(0, description="tokens consumed by judge")

# === File: backend/app/utils/__init__.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""工具包。"""

# === File: backend/app/utils/cost.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""成本计算工具 —— 基于模型和 Token 数量计算 API 调用费用。

用法:
    from app.utils.cost import calculate_cost
    cost = calculate_cost("gpt-4o-mini", prompt_tokens=1000, completion_tokens=500)
"""

from __future__ import annotations

from dataclasses import dataclass

@dataclass(frozen=True)
class ModelPricing:
    """每百万 Token 的价格（USD）。"""

    prompt_per_million: float
    completion_per_million: float

MODEL_PRICING: dict[str, ModelPricing] = {
```

```
    "gpt-4o": ModelPricing(2.50, 10.00),
    "gpt-4o-2024-11-20": ModelPricing(2.50, 10.00),
    "gpt-4o-2024-08-06": ModelPricing(2.50, 10.00),
    "gpt-4o-2024-05-13": ModelPricing(5.00, 15.00),
    "gpt-4o-mini": ModelPricing(0.15, 0.60),
    "gpt-4o-mini-2024-07-18": ModelPricing(0.15, 0.60),
    "gpt-4.1": ModelPricing(2.00, 8.00),
    "gpt-4.1-mini": ModelPricing(0.40, 1.60),
    "gpt-4.1-nano": ModelPricing(0.10, 0.40),
    "o4-mini": ModelPricing(1.10, 4.40),
    "o3": ModelPricing(2.00, 8.00),
    "o3-mini": ModelPricing(1.10, 4.40),
    "o1": ModelPricing(15.00, 60.00),
    "o1-mini": ModelPricing(1.10, 4.40),
    "gpt-4-turbo": ModelPricing(10.00, 30.00),
    "gpt-4-turbo-preview": ModelPricing(10.00, 30.00),
    "gpt-3.5-turbo": ModelPricing(0.50, 1.50),
    "gpt-3.5-turbo-0125": ModelPricing(0.50, 1.50),
}

_DEFAULT_PRICING = ModelPricing(0.15, 0.60)

def calculate_cost(
    model: str,
    prompt_tokens: int,
    completion_tokens: int,
) -> float:
    """计算单次 API 调用的费用（USD）。

    Args:
        model: 模型名称（如 "gpt-4o-mini"）。
        prompt_tokens: 输入 Token 数。
        completion_tokens: 输出 Token 数。

    Returns:
        费用（USD），保留 8 位小数以支持微量计费。

    Examples:
        >>> calculate_cost("gpt-4o-mini", 1000, 500)
        0.00045
        >>> calculate_cost("gpt-4o", 10000, 5000)
        0.075
    """
    pricing = MODEL_PRICING.get(model, _DEFAULT_PRICING)
    cost = (prompt_tokens / 1_000_000) * pricing.prompt_per_million + (
        completion_tokens / 1_000_000
    ) * pricing.completion_per_million
    return round(cost, 8)

def get_supported_models() -> list[str]:
    """返回所有已配置定价的模型列表。"""
    return sorted(MODEL_PRICING.keys())

# === File: backend/app/utils/exceptions.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Custom exception hierarchy and unified error response format."""

from __future__ import annotations
from typing import Any
```

```
from datetime import datetime, timezone

class AgentFlowError(Exception):
    """Base exception for all application errors."""

    def __init__(
        self,
        message: str = "Internal error",
        status_code: int = 500,
        detail: Any = None,
    ):
        self.message = message
        self.status_code = status_code
        self.detail = detail
        super().__init__(self.message)

class NotFoundError(AgentFlowError):
    """Resource not found (HTTP 404)."""

    def __init__(self, resource: str = "Resource", resource_id: str = ""):
        msg = (
            f"{resource} not found: {resource_id}"
            if resource_id
            else f"{resource} not found"
        )
        super().__init__(message=msg, status_code=404)

class ValidationError(AgentFlowError):
    """Request validation failed (HTTP 422)."""

    def __init__(self, message: str = "Validation failed", detail: Any = None):
        super().__init__(message=message, status_code=422, detail=detail)

class BusinessError(AgentFlowError):
    """Business logic error (HTTP 400)."""

    def __init__(self, message: str = "Business error", detail: Any = None):
        super().__init__(message=message, status_code=400, detail=detail)

class TaskStateError(AgentFlowError):
    """Task state conflict (HTTP 409)."""

    def __init__(self, message: str = "Task state error"):
        super().__init__(message=message, status_code=409)

class ExternalServiceError(AgentFlowError):
    """External service unavailable (HTTP 503)."""

    def __init__(self, message: str = "External service error", detail: Any = None):
        super().__init__(message=message, status_code=503, detail=detail)

class RateLimitError(AgentFlowError):
    """Rate limit exceeded (HTTP 429)."""

    def __init__(self, message: str = "Too many requests", detail: Any = None):
        super().__init__(message=message, status_code=429, detail=detail)

class ForbiddenError(AgentFlowError):
    """Permission denied (HTTP 403)."""

    def __init__(self, message: str = "Forbidden", detail: Any = None):
        super().__init__(message=message, status_code=403, detail=detail)

def error_response(
    status_code: int,
    message: str,
    detail: Any = None,
```

```
    request_id: str | None = None,
    stacktrace: str | None = None,
    error_id: str | None = None,
) -> dict[str, Any]:
    """Generate unified JSON error response.

    Returns dict with code, message, detail, timestamp, request_id,
    optional error_id (support correlation), and stacktrace (dev only).
    """
    resp: dict[str, Any] = {
        "error": {
            "code": status_code,
            "message": message,
            "timestamp": datetime.now(timezone.utc).isoformat(),
        }
    }
    if detail:
        resp["error"]["detail"] = detail
    if request_id:
        resp["error"]["request_id"] = request_id
    if error_id:
        resp["error"]["error_id"] = error_id
    if stacktrace:
        resp["error"]["stacktrace"] = stacktrace
    return resp

# === File: backend/app/utils/logger.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Application logging entrypoint — delegates to AOLS (structlog JSON)."""

from __future__ import annotations

from typing import Any

def setup_logging() -> None:
    """Configure process-wide logging (structlog + rotating file)."""
    from app.core.observability.aols.logger import setup_aols_logging

    setup_aols_logging()

def get_logger(name: str | None = None) -> Any:
    """Structured logger factory (compat re-export)."""
    from app.core.observability.aols.logger import get_logger as _get

    return _get(name)

# === File: backend/scripts/backfill_created_by.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
"""One-off SQLite backfill for tasks.created_by."""
from sqlalchemy import create_engine, text

engine = create_engine("sqlite:///./agentflow_eval.db")
with engine.begin() as conn:
    cols = {r[1] for r in conn.execute(text("PRAGMA table_info(tasks)")).fetchall()}
    if "created_by" not in cols:
        conn.execute(
            text(
                "ALTER TABLE tasks ADD COLUMN created_by "
                "VARCHAR(100) NOT NULL DEFAULT 'anonymous'"
            )
        )
        print("added created_by")
    else:
```

```
        print("created_by already present")

# === File: backend/scripts/export_openapi.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
# AgentFlow-Eval V1.0.0
"""Export FastAPI OpenAPI schema and optionally check against frozen contract.

Usage:
  cd backend
  python scripts/export_openapi.py -o ../docs/openapi-v1.json
  python scripts/export_openapi.py --check ../docs/openapi-v1.json
"""

from __future__ import annotations

import argparse
import json
import sys
from pathlib import Path

def _app():
    root = Path(__file__).resolve().parents[1]
    if str(root) not in sys.path:
        sys.path.insert(0, str(root))
    from app.main import app

    return app

def export_schema() -> dict:
    return _app().openapi()

def path_methods(schema: dict) -> set[tuple[str, str]]:
    out: set[tuple[str, str]] = set()
    for path, item in (schema.get("paths") or {}).items():
        if not isinstance(item, dict):
            continue
        for method in item:
            if method.lower() in {
                "get",
                "post",
                "put",
                "patch",
                "delete",
                "head",
                "options",
            }:
                out.add((path, method.lower()))
    return out

def check_compatible(frozen: dict, current: dict) -> list[str]:
    """Return list of breaking-change messages (empty = ok).

    Breaking: frozen (path, method) missing from current.
    Additive endpoints are allowed.
    """
    errors: list[str] = []
    frozen_pm = path_methods(frozen)
    current_pm = path_methods(current)
    missing = frozen_pm - current_pm
    for path, method in sorted(missing):
        errors.append(f"BREAKING: removed {method.upper()} {path}")
    return errors
```

```
def main(argv: list[str] | None = None) -> int:
    parser = argparse.ArgumentParser(description="OpenAPI export / freeze check")
    parser.add_argument(
        "-o",
        "--output",
        type=Path,
        help="Write OpenAPI JSON to this path",
    )
    parser.add_argument(
        "--check",
        type=Path,
        metavar="FROZEN_JSON",
        help="Fail if current schema removes paths from frozen JSON",
    )
    args = parser.parse_args(argv)

    schema = export_schema()
    if args.output:
        args.output.parent.mkdir(parents=True, exist_ok=True)
        args.output.write_text(
            json.dumps(schema, ensure_ascii=False, indent=2) + "\n",
            encoding="utf-8",
        )
        print(f"Wrote {args.output} ({len(path_methods(schema))} operations)")

    if args.check:
        frozen = json.loads(args.check.read_text(encoding="utf-8"))
        errors = check_compatible(frozen, schema)
        if errors:
            print("API compatibility check FAILED:", file=sys.stderr)
            for e in errors:
                print(f"  - {e}", file=sys.stderr)
            return 1
        print(
            f"API compatibility check OK "
            f"(frozen={len(path_methods(frozen))} ops, "
            f"current={len(path_methods(schema))} ops)"
        )

    if not args.output and not args.check:
        json.dump(schema, sys.stdout, ensure_ascii=False, indent=2)
        print()
    return 0

if __name__ == "__main__":
    raise SystemExit(main())

# === File: backend/scripts_local_api_test.py ===
# (c) 2026 AgentFlow-Eval | Author: 李凯昕
"""API functional smoke against local backend + Docker Postgres."""
from __future__ import annotations

import json
import time
import urllib.error
import urllib.request
from pathlib import Path

BASE = "http://127.0.0.1:8000"
```

```
API = BASE + "/api/v1"
results: list[dict] = []

def req(name: str, method: str, path: str, body=None) -> tuple[int, str]:
    if path.startswith("http"):
        url = path
    elif path.startswith("/health") or path.startswith("/metrics"):
        url = BASE + path
    else:
        url = API + path
    data = None
    headers: dict[str, str] = {}
    if body is not None:
        data = json.dumps(body).encode("utf-8")
        headers["Content-Type"] = "application/json"
    request = urllib.request.Request(url, data=data, headers=headers, method=method)
    try:
        with urllib.request.urlopen(request, timeout=90) as resp:
            code = resp.status
            text = resp.read().decode("utf-8", errors="replace")
            ok = 200 <= code < 300
            results.append({"name": name, "code": code, "ok": ok, "detail": text[:200]})
            return code, text
    except urllib.error.HTTPError as e:
        text = e.read().decode("utf-8", errors="replace")
        results.append({"name": name, "code": e.code, "ok": False, "detail": text[:200]})
        return e.code, text
    except Exception as e:  # noqa: BLE001
        results.append({"name": name, "code": 0, "ok": False, "detail": str(e)[:200]})
        return 0, str(e)

def main() -> None:
    for _ in range(30):
        code, _ = req("probe", "GET", "/health/ready")
        if code == 200:
            results.clear()
            break
        time.sleep(0.5)

    checks = [
        ("health", "GET", "/health"),
        ("health_live", "GET", "/health/live"),
        ("health_ready", "GET", "/health/ready"),
        ("metrics", "GET", "/metrics"),
        ("me", "GET", "/me"),
        ("dashboard", "GET", "/dashboard?days=7"),
        ("dashboard_stats", "GET", "/dashboard/stats"),
        ("tasks_list", "GET", "/tasks?page=1&page_size=10"),
        ("tools", "GET", "/tools"),
        ("settings", "GET", "/settings"),
        ("audit", "GET", "/audit?page=1&page_size=5"),
        ("traces", "GET", "/traces?page=1&page_size=5"),
        ("logs", "GET", "/logs?page=1&page_size=10"),
```

```
    ("logs_stats", "GET", "/logs/statistics?days=7"),
    ("obs_kpis", "GET", "/observability/kpis?days=7"),
    ("obs_slow", "GET", "/observability/slow-tasks?limit=10"),
    ("obs_topo", "GET", "/observability/error-topology?days=7"),
    ("diagnosis_list", "GET", "/diagnosis?limit=10"),
    ("plugins", "GET", "/plugins"),
    ("billing_plans", "GET", "/billing/plans"),
    ("billing_quota", "GET", "/billing/quota"),
    ("ab_list", "GET", "/ab"),  # list is GET /ab (not /ab/experiments)
    ("experiments", "GET", "/experiments"),
    ("media_list", "GET", "/media"),
]
for name, method, path in checks:
    req(name, method, path)

code, text = req(
    "task_create",
    "POST",
    "/tasks",
    {
        "name": "容器栈联调任务",
        "description": "Docker Postgres + Celery",
        "agent_config": {
            "model": "gpt-4o-mini",
            "temperature": 0,
            "max_iterations": 3,
        },
    },
)
tid = None
try:
    tid = json.loads(text).get("id")
except Exception:
    tid = None

if tid:
    req(
        "task_suites",
        "POST",
        f"/tasks/{tid}/test-suites",
        [
            {
                "user_query": "1+1等于几？",
                "expected_output": "2",
                "expected_tools": [],
            }
        ],
    )
    req("task_get", "GET", f"/tasks/{tid}")
    req("task_execute", "POST", f"/tasks/{tid}/execute", {})
    for i in range(12):
        time.sleep(5)
```

```
            c, t = req(f"task_poll_{i}", "GET", f"/tasks/{tid}")
            try:
                st = json.loads(t).get("status")
            except Exception:
                st = None
            if st in {"completed", "failed", "cancelled", "timeout"}:
                break
        req("task_report", "GET", f"/reports/{tid}")
        req("diagnosis_task", "GET", f"/diagnosis/{tid}")

    _, text = req("tasks_for_demo", "GET", "/tasks?page=1&page_size=20")
    try:
        items = json.loads(text).get("items") or []
        demo = next((t for t in items if "Demo" in (t.get("name") or "")), None)
        if demo:
            req("diagnosis_demo", "GET", f"/diagnosis/{demo['id']}")
    except Exception:
        pass

    _, text = req("traces_sample", "GET", "/traces?page=1&page_size=1")
    try:
        items = json.loads(text).get("items") or []
        if items:
            req("trace_detail", "GET", f"/traces/{items[0]['id']}")
    except Exception:
        pass

    final = [r for r in results if r["name"] not in {"probe", "tasks_for_demo", "traces_sample"}]
    pass_n = sum(1 for r in final if r["ok"])
    fail_n = sum(1 for r in final if not r["ok"])
    print(f"PASS={pass_n} FAIL={fail_n} TOTAL={len(final)}")
    for r in final:
        flag = "OK" if r["ok"] else "FAIL"
        print(f"{flag:4} {r['code']:3} {r['name']:18} {r['detail'][:90]}")

    out = Path(__file__).resolve().parents[1] / "docs" / "_docker_api_test_results.json"
    out.parent.mkdir(parents=True, exist_ok=True)
    out.write_text(
        json.dumps({"pass": pass_n, "fail": fail_n, "results": final}, ensure_ascii=False, indent=2),
        encoding="utf-8",
    )
    print("wrote", out)

if __name__ == "__main__":
    main()
```